



Lesson 1: Kotlin basics

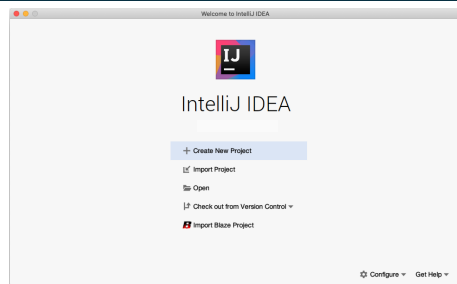
About this lesson

Lesson 1: Kotlin basics

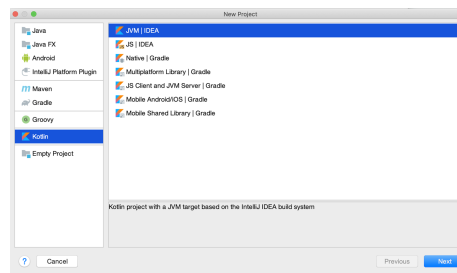
- [Get started](#)
- [Operators](#)
- [Data types](#)
- [Variables](#)
- [Conditionals](#)
- [Lists and arrays](#)
- [Null safety](#)
- [Summary](#)

Get started

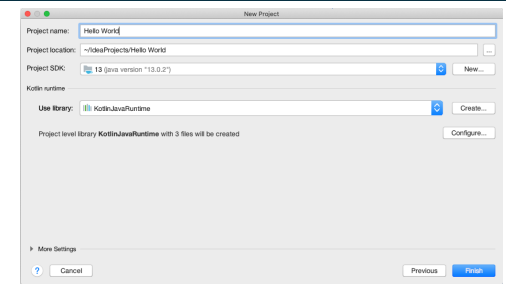
Open IntelliJ IDEA



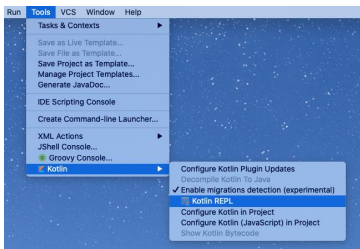
Create a new project



Name the project

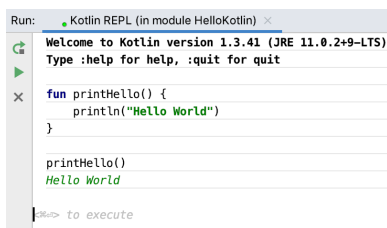


Open REPL (Read-Eval-Print-Loop)



It may take a few moments before the Kotlin menu appears under Tools.

Create a printHello() function



Press **Control+Enter** (Command+Enter on a Mac) to execute.

Operators

Operators

Math operators with integers

Math operators with doubles

- Mathematical operators
 - Increment and decrement operators
 - Comparison operators
 - Assignment operator
 - Equality operators

+ - * / %

++ --

< <= > >=

=

== !=

1 + 1 => 2

53 - 3 => 50

50 / 10 => 5

9 % 3 => 0

1.0 / 2.0 => 0.5

2.0 * 3.5 => 7.0

Math operators

1+1
=> kotlin.Int = 2

53-3
=> kotlin.Int = 50

50/10
=> kotlin.Int = 5

1.0/2.0
=> kotlin.Double = 0.5

2.0*3.5
=> kotlin.Double = 7.0

=> indicates output from your code.
Result includes the type (**kotlin.Int**).

Numeric operator methods

Kotlin keeps numbers as primitives, but lets you call methods on numbers as if they were objects.

2.times(3)
=> kotlin.Int = 6

3.5.plus(4)
=> kotlin.Double = 7.5

2.4.div(2)
=> kotlin.Double = 1.2

Data types

Integer types

Type	Bits	Notes
Long	64	From -2 ⁶³ to 2 ⁶³ -1
Int	32	From -2 ³¹ to 2 ³¹ -1
Short	16	From -32768 to 32767
Byte	8	From -128 to 127

Floating-point and other numeric types

Type	Bits	Notes
Double	64	16 - 17 significant digits
Float	32	6 - 7 significant digits
Char	16	16-bit Unicode character
Boolean	8	True or false. Operations include: - lazy disjunction, && - lazy conjunction, ! - negation

Operand types

Results of operations keep the types of the operands

6*50
=> kotlin.Int = 300

1/2
=> kotlin.Int = 0

6.0*50.0
=> kotlin.Double = 300.0

1.0*2.0
=> kotlin.Double = 0.5

6.0*50
=> kotlin.Double = 300.0

Type casting

Assign an `Int` to a `Byte`

```
val i: Int = 6
val b: Byte = i
println(b)

⇒ error: type mismatch: inferred type is Int but Byte was
expected
```

Convert `Int` to `Byte` with casting

```
val i: Int = 6
println(i.toByte())

⇒ 6
```

Underscores for long numbers

Use underscores to make long numeric constants more readable.

```
val oneMillion = 1_000_000
val idNumber = 999_99_9999L
val hexBytes = 0xFF_EC_DE_5E
val bytes = 0b11010010_01101001_10010100_10010010
```

Strings

Strings are any sequence of characters enclosed by double quotes.

```
val s1 = "Hello world!"
```

String literals can contain escape characters

```
val s2 = "Hello world!\n"
```

Or any arbitrary text delimited by a triple quote (" """)

```
val text = """
var bikes = 50
"""
```

String concatenation

```
val numberOfDogs = 3
val numberOfCats = 2
```

```
"I have $numberOfDogs dogs" + " and $numberOfCats cats"
```

```
⇒ I have 3 dogs and 2 cats
```

String templates

A template expression starts with a dollar sign (\$) and can be a simple value:

```
val i = 10
println("i = $i")

⇒ i = 10
```

Or an expression inside curly braces:

```
val s = "abc"
println("${s.length} is ${s.length}")

⇒ abc.length is 3
```

String template expressions

```
val numberOfShirts = 10
val numberOfPants = 5
```

```
"I have ${numberOfShirts + numberOfPants} items of clothing"
```

```
⇒ I have 15 items of clothing
```

Variables

Variables

- Powerful type inference
 - Let the compiler infer the type
 - You can explicitly declare the type if needed
- Mutable and immutable variables
 - Immutability not enforced, but recommended

Kotlin is a statically-typed language. The type is resolved at compile time and never changes.

Specifying the variable type

Colon Notation

```
var width: Int = 12
var length: Double = 2.5
```

Important: Once a type has been assigned by you or the compiler, you can't change the type or you get an error.

Mutable and immutable variables

- Mutable (Changeable)

```
var score = 10
```

- Immutable (Unchangeable)

```
val name = "Jennifer"
```

Although not strictly enforced, using immutable variables is recommended in most cases.

var and val

```
var count = 1
count = 2

val size = 1
size = 2
=> Error: val cannot be reassigned
```

Conditionals

Control flow

Kotlin features several ways to implement conditional logic:

- If/Else statements
- When statements
- For loops
- While loops

if/else statements

```
val numberOfCups = 30
val numberOfPlates = 50

if (numberOfCups > numberOfPlates) {
    println("Too many cups!")
} else {
    println("Not enough cups!")
}
=> Not enough cups!
```

if statement with multiple cases

```
val guests = 30
if (guests == 0) {
    println("No guests")
} else if (guests < 20) {
    println("Small group of people")
} else {
    println("Large group of people!")
}
=> Large group of people!
```

Ranges

- Data type containing a span of comparable values (e.g., integers from 1 to 100 inclusive)
- Ranges are bounded
- Objects within a range can be mutable or immutable

Ranges in if/else statements

```
val numberOfStudents = 50
if (numberOfStudents in 1..100) {
    println(numberOfStudents)
}
=> 50
```

Note: There are no spaces around the "range to" operator (1..100)

when statement

```
when (results) {
    0 -> println("No results")
    in 1..39 -> println("Got results!")
    else -> println("That's a lot of results!")
}
=> That's a lot of results!
```

As well as a when statement, you can also define a when expression that provides a return value.

for loops

```
val pets = arrayOf("dog", "cat", "canary")
for (element in pets) {
    print(element + " ")
}
⇒ dog cat canary
```

You don't need to define an iterator variable and increment it for each pass.

for loops: elements and indexes

```
for ((index, element) in pets.withIndex()) {
    println("Item at $index is $element\n")
}
⇒ Item at 0 is dog
Item at 1 is cat
Item at 2 is canary
```

for loops: step sizes and ranges

```
for (i in 1..5) print(i)
⇒ 12345

for (i in 5 downTo 1) print(i)
⇒ 54321

for (i in 3..6 step 2) print(i)
⇒ 35

for (i in 'd'..'g') print (i)
⇒ defg
```

while loops

```
var bicycles = 0
while (bicycles < 50) {
    bicycles++
}
println("$bicycles bicycles in the bicycle rack\n")
⇒ 50 bicycles in the bicycle rack

do {
    bicycles--
} while (bicycles > 50)
println("$bicycles bicycles in the bicycle rack\n")
⇒ 49 bicycles in the bicycle rack
```

repeat loops

```
repeat(2) {
    print("Hello!")
}
⇒ Hello!Hello!
```

Lists and arrays

Lists

- Lists are ordered collections of elements
- List elements can be accessed programmatically through their indices
- Elements can occur more than once in a list

An example of a list is a sentence: it's a group of words, their order is important, and they can repeat.

Immutable list using listOf()

Declare a list using `listOf()` and print it out.

```
val instruments = listOf("trumpet", "piano", "violin")
println(instruments)
⇒ [trumpet, piano, violin]
```

Mutable list using mutableListOf()

Lists can be changed using `mutableListOf()`

```
val myList = mutableListOf("trumpet", "piano", "violin")
myList.remove("violin")

⇒ kotlin.Boolean = true
```

With a list defined with `val`, you can't change which list the variable refers to, but you can still change the contents of the list.

Arrays

- Arrays store multiple items
- Array elements can be accessed programmatically through their indices
- Array elements are mutable
- Array size is fixed

Array using arrayOf()

An array of strings can be created using `arrayOf()`

```
val pets = arrayOf("dog", "cat", "canary")
println(java.util.Arrays.toString(pets))
⇒ [dog, cat, canary]
```

With an array defined with `val`, you can't change which array the variable refers to, but you can still change the contents of the array.

Arrays with mixed or single types

An array can contain different types.

```
val mix = arrayOf("hats", 2)
```

An array can also contain just one type (integers in this case).

```
val numbers = intArrayOf(1, 2, 3)
```

Combining arrays

Use the `+` operator.

```
val numbers = intArrayOf(1,2,3)
val numbers2 = intArrayOf(4,5,6)
val combined = numbers2 + numbers
println(Arrays.toString(combined))

⇒ [4, 5, 6, 1, 2, 3]
```

Null safety

Null safety

- In Kotlin, variables cannot be null by default
- You can explicitly assign a variable to null using the safe call operator
- Allow null-pointer exceptions using the `!!` operator
- You can test for null using the elvis (`? :`) operator

Variables cannot be null

In Kotlin, `null` variables are not allowed by default.

Declare an `Int` and assign `null` to it.

```
var numberOfBooks: Int = null
⇒ error: null can not be a value of a non-null type Int
```

Safe call operator

The safe call operator (`?`), after the type indicates that a variable can be `null`.

Declare an `Int?` as nullable

```
var numberOfBooks: Int? = null
```

In general, do not set a variable to null as it may have unwanted consequences.

Testing for null

Check whether the `numberOfBooks` variable is not `null`. Then decrement that variable.

```
var numberOfBooks = 6
if (numberOfBooks != null) {
    numberOfBooks = numberOfBooks.dec()
}
```

Now look at the Kotlin way of writing it, using the safe call operator.

```
var numberOfBooks = 6
numberOfBooks = numberOfBooks?.dec()
```

The !! operator

If you're certain a variable won't be null, use !! to force the variable into a non-null type. Then you can call methods/properties on it.

```
val len = s!!.length
```

throws NullPointerException if s is null

Warning: Because !! will throw an exception, it should only be used when it would be exceptional to hold a null value.

Elvis operator

Chain null tests with the ?: operator.

```
numberOfBooks = numberOfBooks?.dec() ?: 0
```

The ?: operator is sometimes called the "Elvis operator," because it's like a smiley on its side with a pompadour hairstyle, like Elvis Presley styled his hair.

Summary

Summary

In Lesson 1, you learned how to:

- Create an IntelliJ IDEA project, opening REPL, and execute a function
- Use operators and numeric operator methods
- Use data types, type casting, strings, and string templates
- Use variables and type inference, and mutable and immutable variables
- Use conditionals, control flow, and looping structures
- Use lists and arrays
- Use Kotlin's null safety features

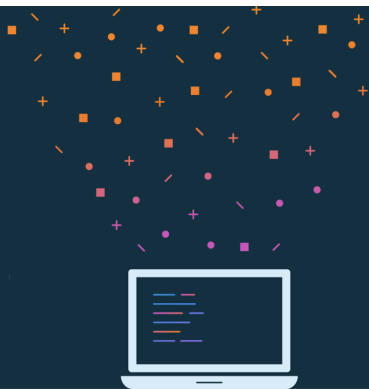
Pathway

Practice what you've learned by completing the pathway:

[Lesson 1: Kotlin basics](#)



Lesson 2: Functions



About this lesson

Lesson 2: Functions

- [Programs in Kotlin](#)
- [\(Almost\) Everything has a value](#)
- [Functions in Kotlin](#)
- [Compact functions](#)
- [Lambdas and higher-order functions](#)
- [List filters](#)
- [Summary](#)

Programs in Kotlin

Setting up

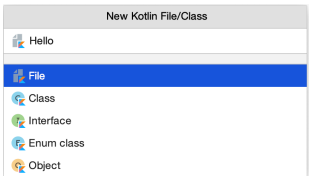
Before you can write code and run programs, you need to:

- Create a file in your project
- Create a main() function
- Pass arguments to main() (Optional)
- Use any passed arguments in function calls (Optional)
- Run your program

Create a new Kotlin file

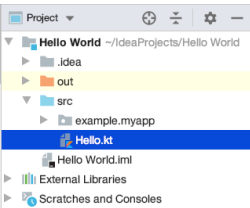
In IntelliJ IDEA's Project pane, under **Hello World**, right-click the `src` folder.

- Select **New > Kotlin File/Class**.
- Select **File**, name the file `Hello`, and press **Enter**.



Create a Kotlin file

You should now see a file in the `src` folder called `Hello.kt`.



Create a main() function

`main()` is the entry point for execution for a Kotlin program.

In the `Hello.kt` file:

```
fun main(args: Array<String>) {  
    println("Hello, world!")  
}
```

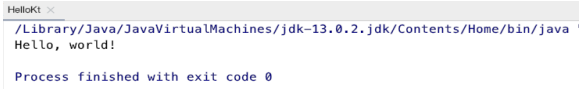
The args in the `main()` function are optional.

Run your Kotlin program

To run your program, click the Run icon (a green play button) to the left of the `main()` function.

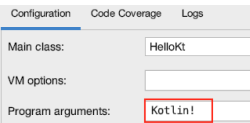
```
1 ▶ fun main(args: Array<String>) {  
2     println("Hello, world!")  
3 }  
4
```

IntelliJ IDEA runs the program, and displays the results in the console.



Pass arguments to main()

Select **Run > Edit Configurations** to open the **Run/Debug Configurations** window.



Use arguments in main()

Use `args[0]` to access the first input argument passed to `main()`.

```
fun main(args: Array<String>) {  
    println("Hello, ${args[0]}")  
}
```

⇒ Hello, Kotlin!

(Almost) Everything has a value

(Almost) Everything is an expression

In Kotlin, almost everything is an expression and has a value. Even an `if` expression has a value.

```
val temperature = 20  
val isHot = if (temperature > 40) true else false  
println(isHot)  
⇒ false
```

Expression values

Sometimes, that value is `kotlin.Unit`.

```
val isUnit = println("This is an expression")  
println(isUnit)  
⇒ This is an expression  
   kotlin.Unit
```

Functions in Kotlin

About functions

- A block of code that performs a specific task
- Breaks a large program into smaller modular chunks
- Declared using the `fun` keyword
- Can take arguments with either named or default values

Parts of a function

Earlier, you created a simple function that printed "Hello World".

```
fun printHello() {  
    println("Hello World")  
}  
  
printHello()
```

Unit returning functions

If a function does not return any useful value, its return type is `Unit`.

```
fun printHello(name: String?): Unit {  
    println("Hi there!")  
}
```

`Unit` is a type with only one value: `Unit`.

Unit returning functions

The `Unit` return type declaration is optional.

```
fun printHello(name: String?): Unit {  
    println("Hi there!")  
}  
  
is equivalent to:  
  
fun printHello(name: String?) {  
    println("Hi there!")  
}
```

Function arguments

Functions may have:

- Default parameters
- Required parameters
- Named arguments

Default parameters

Default values provide a fallback if no parameter value is passed.

```
fun drive(speed: String = "fast") {  
    println("driving $speed")  
}
```

Use "=" after the type to define default values

`drive()` ⇒ driving fast

`drive("slow")` ⇒ driving slow

`drive(speed = "turtle-like")` ⇒ driving turtle-like

Required parameters

If no default is specified for a parameter, the corresponding argument is required.

```
fun tempToday(day: String, temp: Int) {  
    println("Today is $day and it's $temp degrees.")  
}
```

Required parameters

Default versus required parameters

Functions can have a mix of default and required parameters.

```
fun reformat(str: String,  
            divideByCamelHumps: Boolean,  
            wordSeparator: Char,  
            normalizeCase: Boolean = true){
```

Has default value

Pass in required arguments.

`reformat("Today is a day like no other day", false, '_')`

Named arguments

To improve readability, use named arguments for required arguments.

```
reformat(str, divideByCamelHumps = false, wordSeparator = '_')
```

It's considered good style to put default arguments after positional arguments, that way callers only have to specify the required arguments.

Compact functions

Single-expression functions

Compact functions, or single-expression functions, make your code more concise and readable.

```
fun double(x: Int): Int {  
    x * 2  
}  
  
fun double(x: Int): Int = x * 2
```

Complete version

Compact version

Lambdas and higher-order functions

Kotlin functions are first-class

- Kotlin functions can be stored in variables and data structures
- They can be passed as arguments to, and returned from, other higher-order functions
- You can use higher-order functions to create new "built-in" functions

Lambda functions

A lambda is an expression that makes a function that has no name.

```
var dirtLevel = 20  
val waterFilter = {level: Int -> level / 2}  
println(waterFilter(dirtLevel))  
⇒ 10
```

Parameter and type

Function arrow

Code to execute

Syntax for function types

Kotlin's syntax for function types is closely related to its syntax for lambdas. Declare a variable that holds a function.

```
val waterFilter: (Int) -> Int = {level -> level / 2}
```

Variable name

Data type of variable (function type)

Function

Higher-order functions

Higher-order functions take functions as parameters, or return a function.

```
fun encodeMsg(msg: String, encode: (String) -> String): String {  
    return encode(msg)  
}
```

The body of the code calls the function that was passed as the second argument, and passes the first argument along to it.

Higher-order functions

To call this function, pass it a string and a function.

```
val enc1: (String) -> String = { input -> input.toUpperCase() }  
println(encodeMsg("abc", enc1))
```

Using a function type separates its implementation from its usage.

Passing a function reference

Use the `::` operator to pass a named function as an argument to another function.

```
fun enc2(input:String): String = input.reversed()

encodeMessage("abc", ::enc2)
```

Passing a named function, not a lambda

The `::` operator lets Kotlin know that you are passing the function reference as an argument, and not trying to call the function.

Last parameter call syntax

Kotlin prefers that any parameter that takes a function is the last parameter.

```
encodeMessage("acronym", { input -> input.toUpperCase() })
```

You can pass a lambda as a function parameter without putting it inside the parentheses.

```
encodeMsg("acronym") { input -> input.toUpperCase() }
```

Using higher-order functions

Many Kotlin built-in functions are defined using last parameter call syntax.

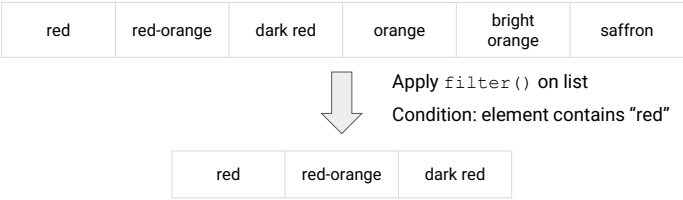
```
inline fun repeat(times: Int, action: (Int) -> Unit)

repeat(3) {
    println("Hello")
}
```

List filters

List filters

Get part of a list based on some condition



Iterating through lists

If a function literal has only one parameter, you can omit its declaration and the `->`. The parameter is implicitly declared under the name `it`.

```
val ints = listOf(1, 2, 3)
ints.filter { it > 0 }
```

Filter iterates through a collection, where `it` is the value of the element during the iteration. This is equivalent to:

```
ints.filter { n: Int -> n > 0 } OR ints.filter { n -> n > 0 }
```

List filters

The filter condition in curly braces `{ }` tests each item as the filter loops through. If the expression returns `true`, the item is included.

```
val books = listOf("nature", "biology", "birds")
println(books.filter { it[0] == 'b' })
⇒ [biology, birds]
```

Eager and lazy filters

Evaluation of expressions in lists:

- Eager:** occurs regardless of whether the result is ever used
- Lazy:** occurs only if necessary at runtime

Lazy evaluation of lists is useful if you don't need the entire result, or if the list is exceptionally large and multiple copies wouldn't fit into RAM.

Eager filters

Filters are eager by default. A new list is created each time you use a filter.

```
val instruments = listOf("viola", "cello", "violin")
val eager = instruments.filter { it [0] == 'v' }
println("eager: " + eager)
⇒ eager: [viola, violin]
```

Lazy filters

Sequences are data structures that use lazy evaluation, and can be used with filters to make them lazy.

```
val instruments = listOf("viola", "cello", "violin")
val filtered = instruments.asSequence().filter { it[0] == 'v' }
println("filtered: " + filtered)
⇒ filtered: kotlin.sequences.FilteringSequence@386cc1c4
```

Sequences -> lists

Sequences can be turned back into lists using `toList()`.

```
val filtered = instruments.asSequence().filter { it[0] == 'v' }
val newList = filtered.toList()
println("new list: " + newList)
⇒ new list: [viola, violin]
```

Other list transformations

- `map()` performs the same transform on every item and returns the list.

```
val numbers = setOf(1, 2, 3)
println(numbers.map { it * 3 })
⇒ [3, 6, 9]
```

- `flatten()` returns a single list of all the elements of nested collections.

```
val numberSets = listOf(setOf(1, 2, 3), setOf(4, 5), setOf(1, 2))
println(numberSets.flatten())
⇒ [1, 2, 3, 4, 5, 1, 2]
```

Summary

Summary

In Lesson 2, you learned how to:

- Create a file and a `main()` function in your project, and run a program
- Pass arguments to the `main()` function
- Use the returned value of an expression
- Use default arguments to replace multiple versions of a function
- Use compact functions, to make code more readable
- Use lambdas and higher-order functions
- Use eager and lazy list filters

Pathway

Practice what you've learned by completing the pathway:

[Lesson 2: Functions](#)



Lesson 3: Classes and objects

About this lesson

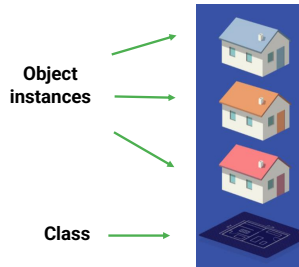
Lesson 3: Classes and objects

- [Classes](#)
- [Inheritance](#)
- [Extension functions](#)
- [Special classes](#)
- [Organizing your code](#)
- [Summary](#)

Classes

Class

- Classes are blueprints for objects
- Classes define methods that operate on their object instances



Class versus object instance

House Class

Data

- House color (String)
- Number of windows (Int)
- Is for sale (Boolean)

Behavior

- `updateColor()`
- `putOnSale()`

Object Instances



Define and use a class

Class Definition

```
class House {  
    val color: String = "white"  
    val numberOfWindows: Int = 2  
    val isForSale: Boolean = false  
  
    fun updateColor(newColor: String){...}  
    ...  
}
```

Create New Object Instance

```
val myHouse = House()  
println(myHouse)
```

Constructors

When a constructor is defined in the class header, it can contain:

- No parameters
`class A`
- Parameters
 - Not marked with `var` or `val` → copy exists only within scope of the constructor
`class B(x: Int)`
 - Marked `var` or `val` → copy exists in all instances of the class
`class C(val y: Int)`

Constructor examples

```
class A
```

```
val aa = A()
```

```
class B(x: Int)
```

```
val bb = B(12)  
println(bb.x)  
=> compiler error unresolved  
reference
```

```
class C(val y: Int)
```

```
val cc = C(42)  
println(cc.y)  
=> 42
```

Default parameters

Class instances can have default values.

- Use default values to reduce the number of constructors needed
- Default parameters can be mixed with required parameters
- More concise (don't need to have multiple constructor versions)

```
class Box(val length: Int, val width: Int = 20, val height: Int = 40)  
val box1 = Box(100, 20, 40)  
val box2 = Box(length = 100)  
val box3 = Box(length = 100, width = 20, height = 40)
```

Primary constructor

Declare the primary constructor within the class header.

```
class Circle(i: Int) {  
    init {  
        ...  
    }  
}
```

This is technically equivalent to:

```
class Circle {  
    constructor(i: Int) {  
        ...  
    }  
}
```

Initializer block

- Any required initialization code is run in a special `init` block
- Multiple `init` blocks are allowed
- `init` blocks become the body of the primary constructor

Initializer block example

Use the `init` keyword:

```
class Square(val side: Int) {  
    init {  
        println(side * 2)  
    }  
}
```

```
val s = Square(10)  
=> 20
```

Multiple constructors

- Use the `constructor` keyword to define secondary constructors
- Secondary constructors must call:
 - The primary constructor using `this` keyword
 - OR
 - Another secondary constructor that calls the primary constructor
- Secondary constructor body is not required

Multiple constructors example

```
class Circle(val radius:Double) {
    constructor(name:String) : this(1.0)
    constructor(diameter:Int) : this(diameter / 2.0) {
        println("in diameter constructor")
    }
    init {
        println("Area: ${Math.PI * radius * radius}")
    }
}
val c = Circle(3)
```

Properties

- Define properties in a class using `val` or `var`
- Access these properties using `dot .` notation with property name
- Set these properties using `dot .` notation with property name (only if declared a `var`)

Person class with name property

```
class Person(var name: String)
fun main() {
    val person = Person("Alex")
    println(person.name) ← Access with .<property name>
    person.name = "Joey" ← Set with .<property name>
    println(person.name)
}
```

Custom getters and setters

If you don't want the default `get/set` behavior:

- Override `get()` for a property
- Override `set()` for a property (if defined as a `var`)

Format: `var` `propertyName`: `DataType` = `initialValue`

```
get() = ...
set(value) {
    ...
}
```

Custom getter

```
class Person(val firstName: String, val lastName:String) {
    val fullName:String
    get() {
        return "$firstName $lastName"
    }
}

val person = Person("John", "Doe")
println(person.fullName)
=> John Doe
```

Custom setter

```
var fullName:String = ""
get() = "$firstName $lastName"
set(value) {
    val components = value.split(" ")
    firstName = components[0]
    lastName = components[1]
    field = value
}

person.fullName = "Jane Smith"
```

Member functions

- Classes can also contain functions
- Declare functions as shown in *Functions* in Lesson 2
 - `fun` keyword
 - Can have default or required parameters
 - Specify return type (if not `Unit`)

Inheritance

Inheritance

- Kotlin has single-parent class inheritance
- Each class has exactly one parent class, called a superclass
- Each subclass inherits all members of its superclass including ones that the superclass itself has inherited

If you don't want to be limited by only inheriting a single class, you can define an interface since you can implement as many of those as you want.

Interfaces

- Provide a contract all implementing classes must adhere to
- Can contain method signatures and property names
- Can derive from other interfaces

Format: `interface` NameOfInterface { interfaceBody }

Interface example

```
interface Shape {
    fun computeArea() : Double
}

class Circle(val radius:Double) : Shape {
    override fun computeArea() = Math.PI * radius * radius
}

val c = Circle(3.0)
println(c.computeArea())
=> 28.274333882308138
```

Extending classes

To extend a class:

- Create a new class that uses an existing class as its core (subclass)
- Add functionality to a class without creating a new one (extension functions)

Creating a new class

- Kotlin classes by default are not subclassable
- Use `open` keyword to allow subclassing
- Properties and functions are redefined with the `override` keyword

Classes are final by default

Declare a class

```
class A
```

Try to subclass A

```
class B : A
```

=>Error: A is final and cannot be inherited from

Use open keyword

Use `open` to declare a class so that it can be subclassed.

Declare a class

```
open class C
```

Subclass from C

```
class D : C()
```

Overriding

- Must use `open` for properties and methods that can be overridden (otherwise you get compiler error)
- Must use `override` when overriding properties and methods
- Something marked `override` can be overridden in subclasses (unless marked `final`)

Abstract classes

- Class is marked as `abstract`
- Cannot be instantiated, must be subclassed
- Similar to an interface with the added the ability to store state
- Properties and functions marked with `abstract` must be overridden
- Can include non-abstract properties and functions

Example abstract classes

```
abstract class Food {
    abstract val kcal : Int
    abstract val name : String
    fun consume() = println("I'm eating ${name}")
}
class Pizza() : Food() {
    override val kcal = 600
    override val name = "Pizza"
}
fun main() {
    Pizza().consume()    // "I'm eating Pizza"
}
```

When to use each

- Defining a broad spectrum of behavior or types? Consider an interface.
- Will the behavior be specific to that type? Consider a class.
- Need to inherit from multiple classes? Consider refactoring code to see if some behavior can be isolated into an interface.
- Want to leave some properties / methods abstract to be defined by subclasses? Consider an abstract class.
- You can extend only one class, but implement one or more interfaces.

Extension functions

Extension functions

Add functions to an existing class that you cannot modify directly.

- Appears as if the implementer added it
- Not actually modifying the existing class
- Cannot access private instance variables

Format: `fun ClassName.functionName( params ) { body }`

Why use extension functions?

- Add functionality to classes that are not open
- Add functionality to classes you don't own
- Separate out core API from helper methods for classes you own

Define extension functions in an easily discoverable place such as in the same file as the class, or a well-named function.

Extension function example

Add `isOdd()` to `Int` class:

```
fun Int.isOdd(): Boolean { return this % 2 == 1 }
```

Call `isOdd()` on an `Int`:

```
3.isOdd()
```

Extension functions are very powerful in Kotlin!

Special classes

Data class

- Special class that exists just to store a set of data
- Mark the class with the `data` keyword
- Generates getters for each property (and setters for vars too)
- Generates `toString()`, `equals()`, `hashCode()`, `copy()` methods, and destructuring operators

Format: `data class <NameOfClass>( parameterList )`

Data class example

Define the data class:

```
data class Player(val name: String, val score: Int)
```

Use the data class:

```
val firstPlayer = Player("Lauren", 10)
println(firstPlayer)
=> Player(name=Lauren, score=10)
```

Data classes make your code much more concise!

Pair and Triple

- Pair and Triple are predefined data classes that store 2 or 3 pieces of data respectively
- Access variables with .first, .second, .third respectively
- Usually named data classes are a better option (more meaningful names for your use case)

Pair and Triple examples

```
val bookAuthor = Pair("Harry Potter", "J.K. Rowling")
println(bookAuthor)
=> (Harry Potter, J.K. Rowling)

val bookAuthorYear = Triple("Harry Potter", "J.K. Rowling", 1997)
println(bookAuthorYear)
println(bookAuthorYear.third)
=> (Harry Potter, J.K. Rowling, 1997)
1997
```

Pair to

Pair's special to variant lets you omit parentheses and periods (infix function). It allows for more readable code

```
val bookAuth1 = "Harry Potter".to("J. K. Rowling")
val bookAuth2 = "Harry Potter" to "J. K. Rowling"
=> bookAuth1 and bookAuth2 are Pair (Harry Potter, J. K. Rowling)
```

Also used in collections like Map and HashMap

```
val map = mapOf(1 to "x", 2 to "y", 3 to "zz")
=> map of Int to String {1=x, 2=y, 3=zz}
```

Enum class

User-defined data type for a set of named values

- Use this to require instances be one of several constant values
- The constant value is, by default, not visible to you
- Use enum before the class keyword

Format: enum class EnumName { NAME1, NAME2, ... NAMEn }

Referenced via EnumName.<ConstantName>

Enum class example

```
Define an enum with red, green, and blue colors.

enum class Color(val r: Int, val g: Int, val b: Int) {
    RED(255, 0, 0), GREEN(0, 255, 0), BLUE(0, 0, 255)
}

println("r: " + Color.RED.r + " g: " + Color.RED.g + " b: " + Color.RED.b)
=> 255 0 0
```

Object/singleton

- Sometimes you only want one instance of a class to ever exist
- Use the object keyword instead of the class keyword
- Accessed with NameOfObject.<function or variable>

Object/singleton example

```
object Calculator {
    fun add(n1: Int, n2: Int): Int {
        return n1 + n2
    }
}

println(Calculator.add(2,4))
=> 6
```

Companion objects

- Lets all instances of a class share a single instance of a set of variables or functions
- Use companion keyword
- Referenced via ClassName.PropertyOrFunction

Companion object example

```
class PhysicsSystem {
    companion object WorldConstants {
        val gravity = 9.8
        val unit = "metric"
        fun computeForce(mass: Double, accel: Double): Double {
            return mass * accel
        }
    }
}

println(PhysicsSystem.WorldConstants.gravity)
println(PhysicsSystem.WorldConstants.computeForce(10.0, 10.0))
=> 9.8100.0
```

Organizing your code

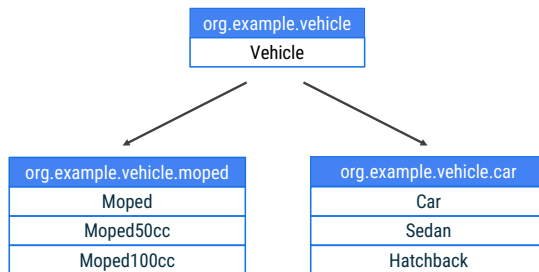
Single file, multiple entities

- Kotlin DOES NOT enforce a single entity (class/interface) per file convention
- You can and should group related structures in the same file
- Be mindful of file length and clutter

Packages

- Provide means for organization
 - Identifiers are generally lower case words separated by periods
 - Declared in the first non-comment line of code in a file following the `package` keyword
- ```
package org.example.game
```

## Example class hierarchy



## Visibility modifiers

Use visibility modifiers to limit what information you expose.

- `public` means visible outside the class. Everything is public by default, including variables and methods of the class.
- `private` means it will only be visible in that class (or source file if you are working with functions).
- `protected` is the same as `private`, but it will also be visible to any subclasses.

## Summary

## Summary

In Lesson 3, you learned about:

- Classes, constructors, and getters and setters
- Inheritance, interfaces, and how to extend classes
- Extension functions
- Special classes: data classes, enums, object/singletons, companion objects
- Packages
- Visibility modifiers

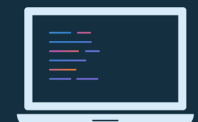
## Pathway

Practice what you've learned by completing the pathway:

[Lesson 3: Classes and objects](#)



Lesson 4:  
Build your first  
Android app



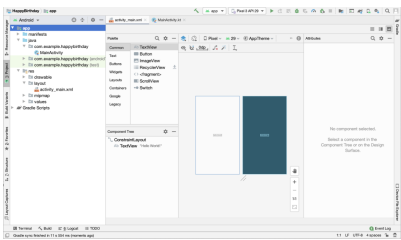
# About this lesson

## Lesson 4: Build your first Android app

- [Your first app](#)
- [Anatomy of an Android app](#)
- [Layouts and resources in Android](#)
- [Activities](#)
- [Make an app interactive](#)
- [Gradle: Building an Android app](#)
- [Accessibility](#)
- [Summary](#)

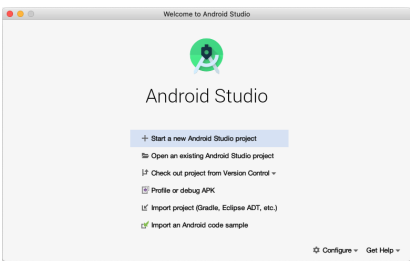
# Android Studio

Official IDE for building Android apps

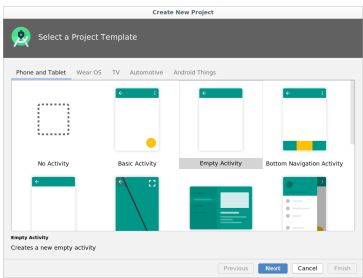


# Your first app

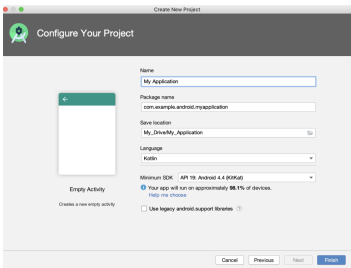
## Open Android Studio



## Create new project



## Enter your project details



## Android releases and API levels

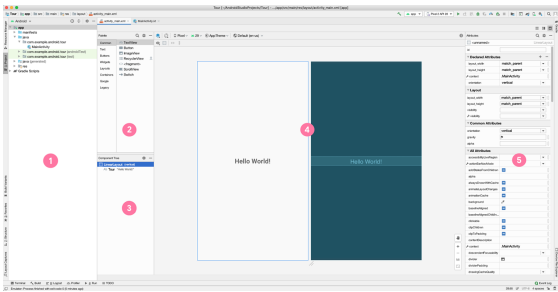
| Platform Version | API Level | VERSION_CODE |
|------------------|-----------|--------------|
| Android 10.0     | 29        | Q            |
| Android 9        | 28        | P            |
| Android 8.1      | 27        | O_MR1        |
| Android 8.0      | 26        | O            |
| Android 7.1.1    | 25        | N_MR1        |
| Android 7.1      | 25        | N_MR1        |
| Android 7.0      | 24        | N            |
| Android 6.0      | 23        | M            |
| Android 5.1      | 22        | LOLLIPOP_MR1 |
| Android 5.0      | 21        | LOLLIPOP     |

## Choose API levels for your app

- Minimum SDK: Device needs at least this API level to install
  - Target SDK: API version and highest Android version tested
  - Compile SDK: Android OS library version compiled with
- ```
minSdkVersion <= targetSdkVersion <= compileSdkVersion
```

The API level identifies the framework API version of the Android SDK.

Tour of Android Studio



Run your app



- Android device (phone, tablet)
- Emulator on your computer

Android Virtual Device (AVD) Manager

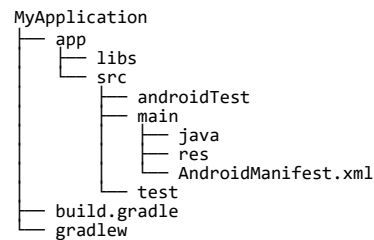


Anatomy of an Android App project

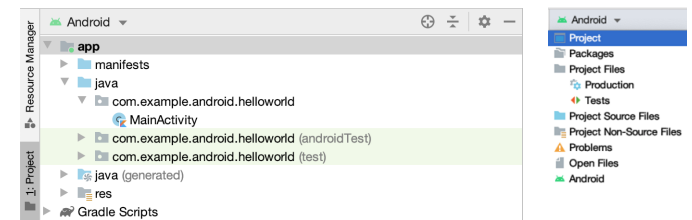
Anatomy of a basic app project

- Activity
- Resources (layout files, images, audio files, themes, and colors)
- Gradle files

Android app project structure



Browse files in Android Studio

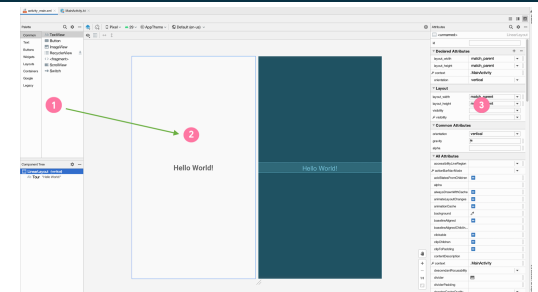


Layouts and resources in Android

Views

- Views are the user interface building blocks in Android
 - Bounded by a rectangular area on the screen
 - Responsible for drawing and event handling
 - Examples: TextView, ImageView, Button
- Can be grouped to form more complex user interfaces

Layout Editor



XML Layouts

You can also edit your layout in XML.

- Android uses XML to specify the layout of user interfaces (including View attributes)
- Each View in XML corresponds to a class in Kotlin that controls how that View functions

XML for a TextView

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"/>
```

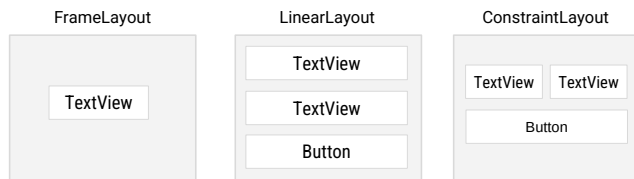
Hello World!

Size of a View

- wrap_content
 android:layout_width="wrap_content"
- match_parent
 android:layout_width="match_parent"
- Fixed value (use dp units)
 android:layout_width="48dp"

ViewGroups

A ViewGroup is a container that determines how views are displayed.

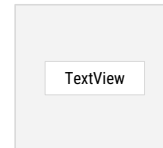


The ViewGroup is the parent and the views inside it are its children.

FrameLayout example

A **FrameLayout** generally holds a single child View.

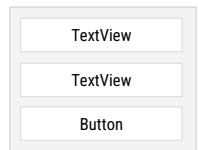
```
<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:text="Hello World!"/>
</FrameLayout>
```



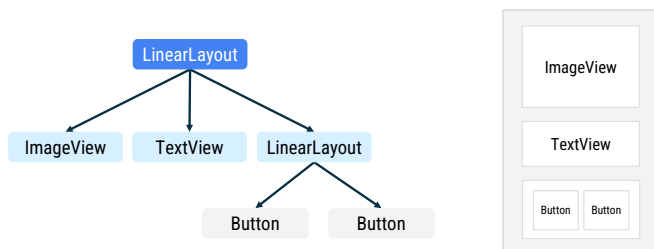
LinearLayout example

- Aligns child views in a row or column
- Set **android:orientation** to horizontal or vertical

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <TextView ... />
    <TextView ... />
    <Button ... />
</LinearLayout>
```



View hierarchy



App resources

Static content or additional files that your code uses

- Layout files
- Images
- Audio files
- User interface strings
- App icon

Common resource directories

Add resources to your app by including them in the appropriate resource directory under the parent **res** folder.

```
main
├── java
├── res
│   ├── drawable
│   ├── layout
│   ├── mipmap
│   └── values
```

Resource IDs

- Each resource has a resource ID to access it.
 - When naming resources, the convention is to use all lowercase with underscores (for example, `activity_main.xml`).
 - Android autogenerates a class file named `R.java` with references to all resources in the app.
 - Individual items are referenced with:
`R.<resource_type>.<resource_name>`
- Examples: `R.drawable.ic_launcher` (`res/drawable/ic_launcher.xml`)
`R.layout.activity_main` (`res/layout/activity_main.xml`)

Resource IDs for views

Individual views can also have resource IDs.

Add the `android:id` attribute to the View in XML. Use `@+id/name` syntax.

```
<TextView
    android:id="@+id/helloTextView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"/>
```

Within your app, you can now refer to this specific TextView using:
`R.id.helloTextView`

Activities

What's an Activity?

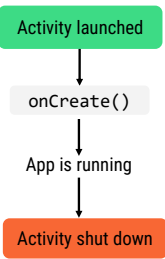


- An Activity is a means for the user to accomplish one main goal.
- An Android app is composed of one or more activities.

MainActivity.kt

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

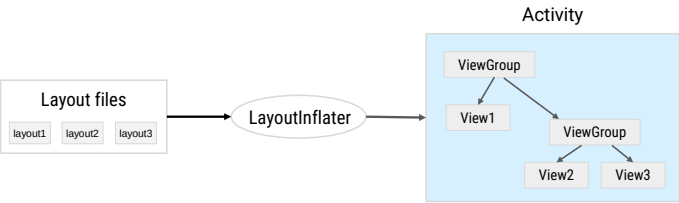
How an Activity runs



Implement the onCreate() callback

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
}
```

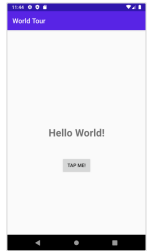
Layout inflation



Make an app interactive

Define app behavior in Activity

Modify the Activity so the app responds to user input, such as a button tap.



Modify a View dynamically

Within `MainActivity.kt`:

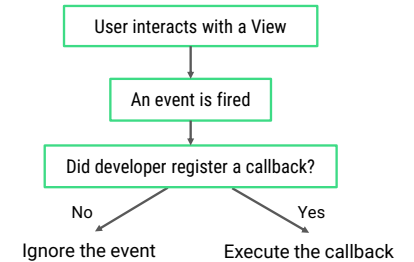
Get a reference to the View in the view hierarchy:

```
val resultTextView: TextView = findViewById(R.id.textView)
```

Change properties or call methods on the View instance:

```
resultTextView.text = "Goodbye!"
```

Set up listeners for specific events



View.OnClickListener

```
class MainActivity : AppCompatActivity(), View.OnClickListener {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        ...  
        val button: Button = findViewById(R.id.button)  
        button.setOnClickListener(this)  
    }  
  
    override fun onClick(v: View?) {  
        TODO("not implemented")  
    }  
}
```

SAM (single abstract method)

Converts a function into an implementation of an interface

Format: `InterfaceName { lambda body }`

```
val runnable = Runnable { println("Hi there") }
```

is equivalent to

```
val runnable = (object: Runnable {  
    override fun run() {  
        println("Hi there")  
    }  
})
```

View.OnClickListener as a SAM

A more concise way to declare a click listener

```
class MainActivity : AppCompatActivity() {  
    ...  
    override fun onCreate(savedInstanceState: Bundle?) {  
        ...  
        val button: Button = findViewById(R.id.button)  
        button.setOnClickListener { view -> /* do something */ }  
    }  
}
```

Late initialization

```
class Student(val id: String) {  
    lateinit var records: HashSet<Any>  
  
    init {  
        // retrieve records given an id  
    }  
}
```

Lateinit example in Activity

```
class MainActivity : AppCompatActivity() {  
    lateinit var result: TextView  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        ...  
        result = findViewById(R.id.result_text_view)  
    }  
}
```

Gradle: Building an Android app

What is Gradle?

- Builds automation system
- Manages the build cycle via a series of tasks (for example, compiles Kotlin sources, runs tests, installs app to device)
- Determines the proper order of tasks to run
- Manages dependencies between projects and third-party libraries

Gradle build file

- Declare plugins
- Define Android properties
- Handle dependencies
- Connect to repositories

Plugins

Provide libraries and infrastructure needed by your app

```
apply plugin: 'com.android.application'
apply plugin: 'kotlin-android'
apply plugin: 'kotlin-android-extensions'
```

Android configuration

```
android {
    compileSdkVersion 30
    buildToolsVersion "30.0.2"

    defaultConfig {
        applicationId "com.example.sample"
        minSdkVersion 19
        targetSdkVersion 30
    }
}
```

Dependencies

```
dependencies {
    implementation "org.jetbrains.kotlin:kotlin-stdlib-jdk7:$kotlin_version"
    implementation 'androidx.core:core-ktx:1.3.2'
    implementation 'androidx.appcompat:appcompat:1.2.0'
    implementation 'com.google.android.material:material:1.2.1'
    ...
}
```

Repositories

```
repositories {
    google()
    mavenCentral()
}
```

Common Gradle tasks

- Clean
- Tasks
- InstallDebug

Accessibility

Accessibility

- Refers to improving the design and functionality of your app to make it easier for more people, including those with disabilities, to use
- Making your app more accessible leads to an overall better user experience and benefits all your users

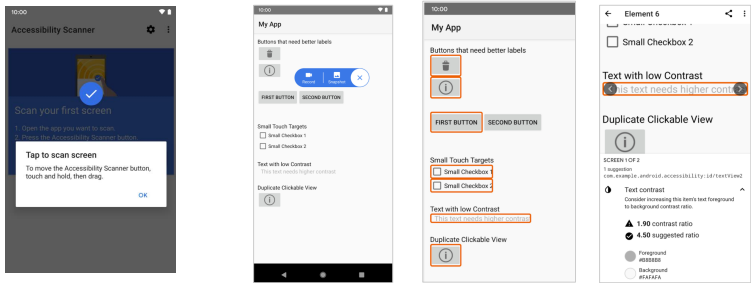
Make apps more accessible

- Increase text visibility with foreground and background color contrast ratio:
 - At least 4.5:1 for small text against the background
 - At least 3.0:1 for large text against the background
- Use large, simple controls
 - Touch target size should be at least 48dp x 48dp
- Describe each UI element
 - Set content description on images and controls

Accessibility Scanner

Tool that scans your screen and suggests improvements to make your app more accessible, based on:

- Content labels
- Touch target sizes
- Clickable views
- Text and image contrast



Accessibility Scanner example

Add content labels

- Set `contentDescription` attribute → read aloud by screen reader
- ```
<ImageView
...
 android:contentDescription="@string/stop_sign" />
```
- Text in `TextView` already provided to accessibility services, no additional label needed

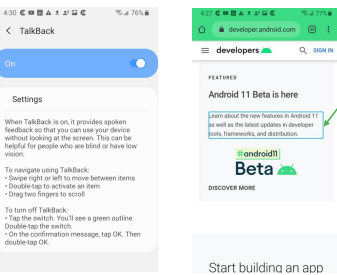
# No content label needed

- For graphical elements that are purely for decorative purposes, you can set `android:importantForAccessibility="no"`
- Removing unnecessary announcements is better for the user

# TalkBack

- Google screen reader included on Android devices
- Provides spoken feedback so you don't have to look at the screen to use your device
- Lets you navigate the device using gestures
- Includes braille keyboard for Unified English Braille

# TalkBack example



Reads text aloud as user navigates the screen

# Switch access

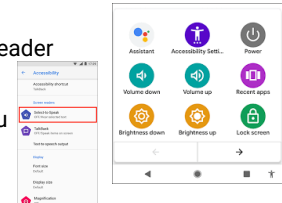
- Allows for controlling the device using one or more switches instead of the touchscreen
- Switch Access will let users move through menus, home screen icons, or other items and select the appropriate action
- Scans your app UI and highlights each item until you make a selection
- Use with external switch, external keyboard, or buttons on the Android device (e.g., volume buttons)



# Android Accessibility Suite

Collection of accessibility apps that help you use your Android device eyes-free, or with a switch device. It includes:

- Talkback screen reader
- Switch Access
- Accessibility Menu
- Select to Speak



# Accessibility Resources

- [Build more accessible apps](#)
- [Principles for improving app accessibility](#)
- [Basic Android Accessibility codelab](#)
- [Material Design best practices on accessibility](#)

# Summary

# Summary

- In Lesson 4, you learned how to:
- Use Views and ViewGroups to build the user interface of your app
  - Access resources in your app from `R.<resource_type>.<resource_name>`
  - Define app behavior in the Activity (for example, register `OnClickListener`)
  - Use Gradle as the build system to build your app
  - Follow best practices to make your apps more accessible

# Learn more

- [Layouts](#)
- [LinearLayout](#)
- [Input events overview](#)
- [View](#)
- [ViewGroup](#)



## Lesson 5: Layouts

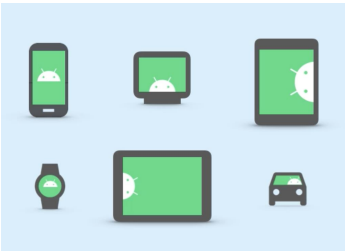
# About this lesson

- Lesson 5: Layouts
- [Layouts in Android](#)
  - [ConstraintLayout](#)
  - [Additional topics for ConstraintLayout](#)
  - [Data binding](#)
  - [Displaying lists with RecyclerView](#)
  - [Summary](#)

# Layouts in Android

## Android devices

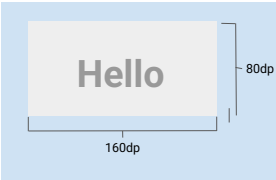
- Android devices come in many different form factors.
- More and more pixels per inch are being packed into device screens.
- Developers need the ability to specify layout dimensions that are consistent across devices.



## Density-independent pixels (dp)

Use dp when specifying sizes in your layout, such as the width or height of views.

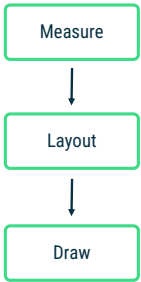
- Density-independent pixels (dp) take screen density into account.
- Android views are measured in density-independent pixels.
- $dp = \frac{\text{width in pixels} \times 160}{\text{screen density}}$



## Screen-density buckets

| Density qualifier       | Description                    | DPI estimate |
|-------------------------|--------------------------------|--------------|
| ldpi (mostly unused)    | Low density                    | ~120dpi      |
| mdpi (baseline density) | Medium density                 | ~160dpi      |
| hdpi                    | High density                   | ~240dpi      |
| xhdpi                   | Extra-high density             | ~320dpi      |
| xxhdpi                  | Extra-extra-high density       | ~480dpi      |
| xxxhdpi                 | Extra-extra-extra-high density | ~640dpi      |

## Android View rendering cycle

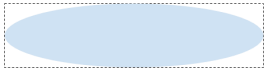


## Drawing region

What we see:

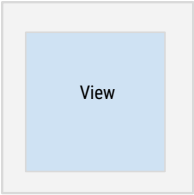


How it's drawn:

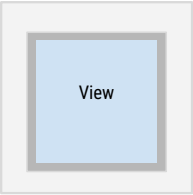


## View margins and padding

View with margin



View with margin and padding



# ConstraintLayout

## Deeply nested layouts are costly

- Deeply nested ViewGroups require more computation
- Views may be measured multiple times
- Can cause UI slowdown and lack of responsiveness

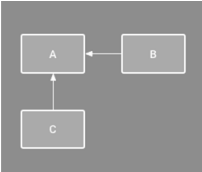
Use ConstraintLayout to avoid some of these issues!

## What is ConstraintLayout?

- Recommended default layout for Android
- Solves costly issue of too many nested layouts, while allowing complex behavior
- Position and size views within it using a set of constraints

## What is a constraint?

A restriction or limitation on the properties of a View that the layout attempts to respect



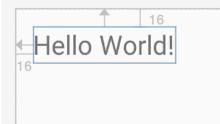
## Relative positioning constraints

Can set up a constraint relative to the parent container

**Format:**  
`layout_constraint<SourceConstraint>_to<TargetConstraint>Of`

Example attributes on a TextView:

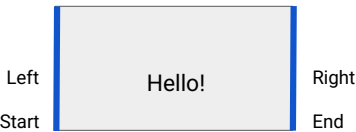
```
app:layout_constraintTop_toTopOf="parent"
app:layout_constraintLeft_toLeftOf="parent"
```



## Relative positioning constraints



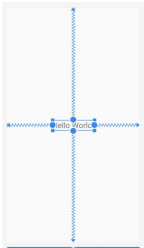
## Relative positioning constraints



## Simple ConstraintLayout example

```
<androidx.constraintlayout.widget.ConstraintLayout
 android:layout_width="match_parent"
 android:layout_height="match_parent">

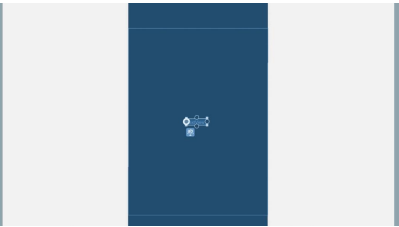
 <TextView
 ...
 app:layout_constraintBottom_toBottomOf="parent"
 app:layout_constraintEnd_toEndOf="parent"
 app:layout_constraintStart_toStartOf="parent"
 app:layout_constraintTop_toTopOf="parent" />
```



```
</androidx.constraintlayout.widget.ConstraintLayout>
```

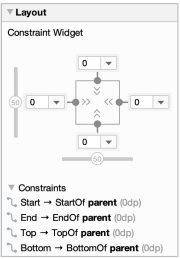
## Layout Editor in Android Studio

You can click and drag to add constraints to a View.

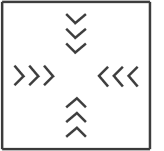
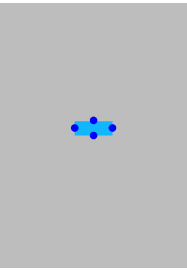


## Constraint Widget in Layout Editor

- Fixed
- Wrap content
- Match constraints

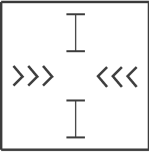
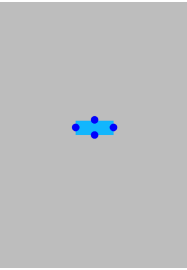


## Wrap content for width and height



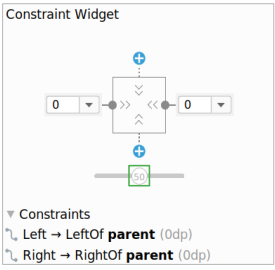
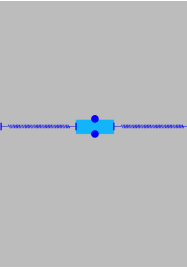
layout\_width wrap\_content  
layout\_height wrap\_content

## Wrap content for width, fixed height



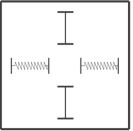
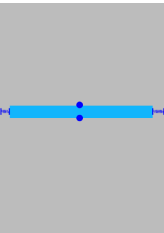
layout\_width wrap\_content  
layout\_height 48dp

## Center a view horizontally



## Use match\_constraint

Can't use match\_parent on a child view, use match\_constraint instead



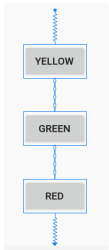
layout\_width 0dp(match\_constraint)  
layout\_height 48dp

# Chains

- Let you position views in relation to each other
- Can be linked horizontally or vertically
- Provide much of LinearLayout functionality

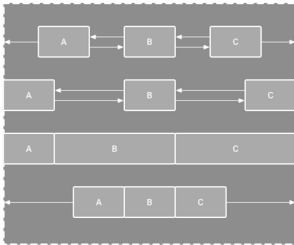
# Create a Chain in Layout Editor

1. Select the objects you want to be in the chain.
2. Right-click and select **Chains**.
3. Create a horizontal or vertical chain.



# Chain styles

Adjust space between views with these different chain styles.



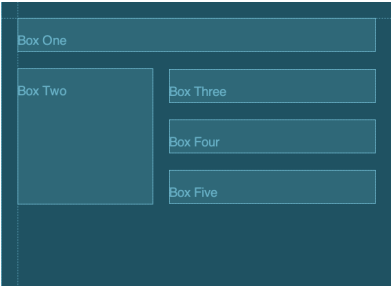
- Spread Chain
- Spread Inside Chain
- Weighted Chain
- Packed Chain

# Additional topics for ConstraintLayout

# Guidelines

- Let you position multiple views relative to a single guide
- Can be vertical or horizontal
- Allow for greater collaboration with design/UX teams
- Aren't drawn on the device

# Guidelines in Android Studio



# Example Guideline

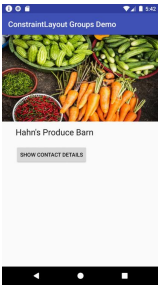
```
<ConstraintLayout>
 <androidx.constraintlayout.widget.Guideline
 android:id="@+id/start_guideline"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:orientation="vertical"
 app:layout_constraintGuide_begin="16dp" />
 <TextView ...
 app:layout_constraintStart_toEndOf="@id/start_guideline" />
</ConstraintLayout>
```

# Creating Guidelines

- layout\_constraintGuide\_begin
- layout\_constraintGuide\_end
- layout\_constraintGuide\_percent

# Groups

- Control the visibility of a set of widgets
- Group visibility can be toggled in code



## Example group

```
<androidx.constraintlayout.widget.Group
 android:id="@+id/group"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 app:constraint_referenced_ids="locationLabel,locationDetails"/>
```

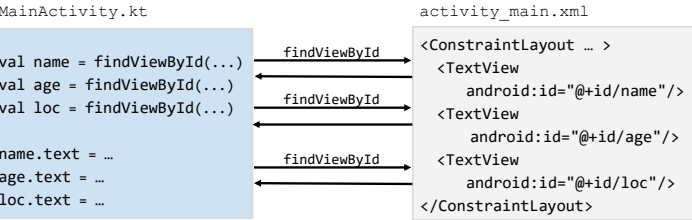
## Groups app code

```
override fun onClick(v: View?) {
 if (group.visibility == View.GONE) {
 group.visibility = View.VISIBLE
 button.setText(R.string.hide_details)
 } else {
 group.visibility = View.GONE
 button.setText(R.string.show_details)
 }
}
```

# Data binding

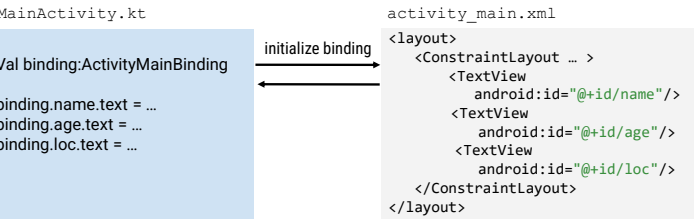
## Current approach: findViewById()

Traverses the View hierarchy each time



## Use data binding instead

Bind UI components in your layouts to data sources in your app.



## Modify build.gradle file

```
android {
 ...
 buildFeatures {
 dataBinding true
 }
}
```

## Add layout tag

```
<layout>
 <androidx.constraintlayout.widget.ConstraintLayout>
 <TextView ... android:id="@+id/username" />
 <EditText ... android:id="@+id/password" />
 </androidx.constraintlayout.widget.ConstraintLayout>
</layout>
```

## Layout inflation with data binding

```
Replace this
setContentView(R.layout.activity_main)

with this
val binding: ActivityMainBinding = DataBindingUtil.setContentView(
 this, R.layout.activity_main)

binding.username = "Melissa"
```

## Data binding layout variables

```
<layout>
 <data>
 <variable name="name" type="String"/>
 </data>
 <androidx.constraintlayout.widget.ConstraintLayout>
 <TextView
 android:id="@+id/textView"
 android:text="@{name}" />
 </androidx.constraintlayout.widget.ConstraintLayout>
</layout>

In MainActivity.kt:
binding.name = "John"
```

## Data binding layout expressions

```
<layout>
 <data>
 <variable name="name" type="String"/>
 </data>

 <androidx.constraintlayout.widget.ConstraintLayout>
 <TextView
 android:id="@+id/textView"
 android:text="@{name.toUpperCase()}" />
 </androidx.constraintlayout.widget.ConstraintLayout>
</layout>
```

## Displaying lists with RecyclerView

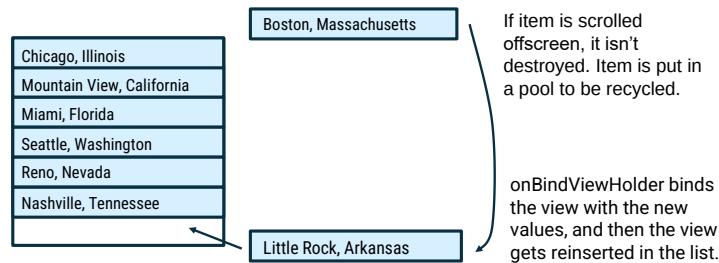
## RecyclerView

- Widget for displaying lists of data
- "Recycles" (reuses) item views to make scrolling more performant
- Can specify a list item layout for each item in the dataset
- Supports animations and transitions

## RecyclerView.Adapter

- Supplies data and layouts that the RecyclerView displays
- A custom Adapter extends from `RecyclerView.Adapter` and overrides these three functions:
  - `getItemCount`
  - `onCreateViewHolder`
  - `onBindViewHolder`

## View recycling in RecyclerView



## Add RecyclerView to your layout

```
<androidx.recyclerview.widget.RecyclerView
 android:id="@+id/rv"
 android:scrollbars="vertical"
 android:layout_width="match_parent"
 android:layout_height="match_parent"/>
```

## Create a list item layout

```
res/layout/item_view.xml

<FrameLayout
 android:layout_width="match_parent"
 android:layout_height="wrap_content">
 <TextView
 android:id="@+id/number"
 android:layout_width="match_parent"
 android:layout_height="wrap_content" />
</FrameLayout>
```

## Create a list adapter

```
class MyAdapter(val data: List<Int>) : RecyclerView.Adapter<MyAdapter.MyViewHolder>() {
 class MyViewHolder(val row: View) : RecyclerView.ViewHolder(row) {
 val textView = row.findViewById<TextView>(R.id.number)
 }

 override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MyViewHolder {
 val layout = LayoutInflater.from(parent.context).inflate(R.layout.item_view,
 parent, false)
 return MyViewHolder(layout)
 }

 override fun onBindViewHolder(holder: MyViewHolder, position: Int) {
 holder.textView.text = data.get(position).toString()
 }

 override fun getItemCount(): Int = data.size
}
```

## Set the adapter on the RecyclerView

```
In MainActivity.kt:
override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 setContentView(R.layout.activity_main)

 val rv: RecyclerView = findViewById(R.id.rv)
 rv.layoutManager = LinearLayoutManager(this)

 rv.adapter = MyAdapter(IntRange(0, 100).toList())
}
```

# Summary

## Summary

In Lesson 5, you learned how to:

- Specify lengths in dp for your layout
- Work with screen densities for different Android devices
- Render Views to the screen of your app
- Layout views within a ConstraintLayout using constraints
- Simplify getting View references from layout with data binding
- Display a list of text items using a RecyclerView and custom adapter

## Learn more

- [Pixel density on Android](#)
- [Spacing](#)
- [Device metrics](#)
- [Type scale](#)
- [Build a Responsive UI with ConstraintLayout](#)
- [Data Binding Library](#)
- [Create dynamic lists with RecyclerView](#)

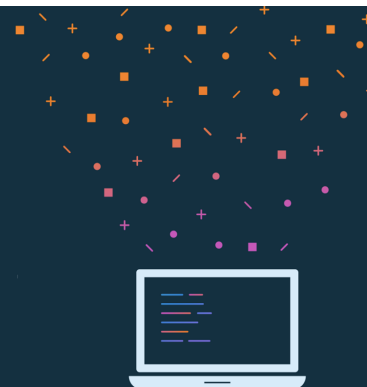
## Pathway

Practice what you've learned by completing the pathway:

[Lesson 5: Layouts](#)



## Lesson 6: App navigation



## About this lesson

Lesson 6: App navigation

- [Multiple activities and intents](#)
- [App bar, navigation drawer, and menus](#)
- [Fragments](#)
- [Navigation in an app](#)
- [More custom navigation behavior](#)
- [Navigation UI](#)
- [Summary](#)

# Multiple activities and intents

## Multiple screens in an app

Sometimes app functionality may be separated into multiple screens.

Examples:

- View details of a single item (for example, product in a shopping app)
- Create a new item (for example, new email)
- Show settings for an app
- Access services in other apps (for example, photo gallery or browse documents)

## Intent

Requests an action from another app component, such as another Activity

- An `Intent` usually has two primary pieces of information:
  - Action to be performed (for example, `ACTION_VIEW`, `ACTION_EDIT`, `ACTION_MAIN`)
  - Data to operate on (for example, a person's record in the contacts database)
- Commonly used to specify a request to transition to another Activity

# Explicit intent

- Fulfills a request **using a specific component**
- Navigates internally to an Activity in your app
- Navigates to a specific third-party app or another app you've written

# Explicit intent examples

Navigate between activities in your app:

```
fun viewNoteDetail() {
 val intent = Intent(this, NoteDetailActivity::class.java)
 intent.putExtra(NOTE_ID, note.id)
 startActivity(intent)
}
```

Navigate to a specific external app:

```
fun openExternalApp() {
 val intent = Intent("com.example.workapp.FILE_OPEN")
 if (intent.resolveActivity(packageManager) != null) {
 startActivity(intent)
 }
}
```

# Implicit intent

- Provides generic action the app can perform
- Resolved using mapping of the data type and action to known components
- Allows any app that matches the criteria to handle the request

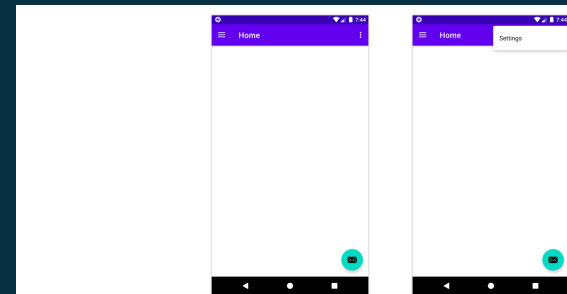
# Implicit intent example

```
fun sendEmail() {
 val intent = Intent(Intent.ACTION_SEND)
 intent.type = "text/plain"
 intent.putExtra(Intent.EXTRA_EMAIL, emailAddresses)
 intent.putExtra(Intent.EXTRA_TEXT, "How are you?")

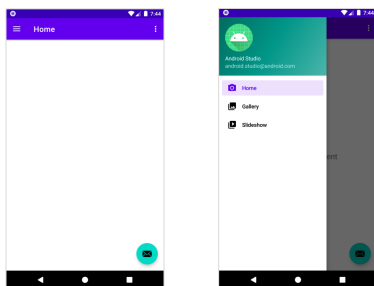
 if (intent.resolveActivity(packageManager) != null) {
 startActivity(intent)
 }
}
```

# App bar, navigation drawer, and menus

# App bar



# Navigation drawer



# Menu

Define menu items in XML menu resource (located in `res/menu` folder)

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
 xmlns:app="http://schemas.android.com/apk/res-auto">
 <item
 android:id="@+id/action_settings"
 android:orderInCategory="100"
 android:title="@string/action_settings"
 app:showAsAction="never" />
</menu>
```

# More menu options

```
<menu>
 <group android:checkableBehavior="single">
 <item
 android:id="@+id/nav_home"
 android:icon="@drawable/ic_menu_camera"
 android:title="@string/menu_home" />
 <item
 android:id="@+id/nav_gallery"
 android:icon="@drawable/ic_menu_gallery"
 android:title="@string/menu_gallery" />
 <item
 android:id="@+id/nav_slideshow"
 android:icon="@drawable/ic_menu_slideshow"
 android:title="@string/menu_slideshow" />
 </group>
</menu>
```

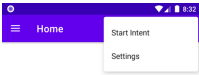
# Options menu example

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
 xmlns:app="http://schemas.android.com/apk/res-auto">

 <item android:id="@+id/action_intent"
 android:title="@string/action_intent" />

 <item
 android:id="@+id/action_settings"
 android:orderInCategory="100"
 android:title="@string/action_settings"
 app:showAsAction="never" />

</menu>
```



# Inflate options menu

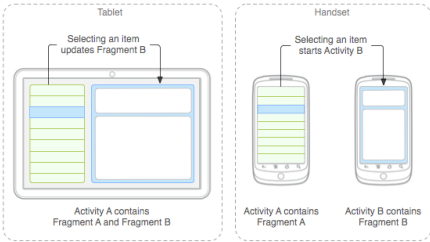
```
override fun onCreateOptionsMenu(menu: Menu): Boolean {
 menuInflater.inflate(R.menu.main, menu)
 return true
}
```

# Handle menu options selected

```
override fun onOptionsItemSelected(item: MenuItem): Boolean {
 when (item.itemId) {
 R.id.action_intent -> {
 val intent = Intent(Intent.ACTION_WEB_SEARCH)
 intent.putExtra(SearchManager.QUERY, "pizza")
 if (intent.resolveActivity(packageManager) != null) {
 startActivity(intent)
 }
 }
 else -> Toast.makeText(this, item.title, Toast.LENGTH_LONG).show()
 }
 ...
}
```

# Fragments

## Fragments for tablet layouts



## Fragment

- Represents a behavior or portion of the UI in an activity ("microactivity")
- Must be hosted in an activity
- Lifecycle tied to host activity's lifecycle
- Can be added or removed at runtime

## Note about fragments

Use the `AndroidX` version of the `Fragment` class. (`androidx.fragment.app.Fragment`).

Don't use the platform version of the `Fragment` class (`android.app.Fragment`), which was deprecated.

# Navigation within an app

## Navigation component

- Collection of libraries and tooling, including an integrated editor, for creating navigation paths through an app
- Assumes one `Activity` per graph with many `Fragment` destinations
- Consists of three major parts:
  - Navigation graph
  - Navigation Host (`NavHost`)
  - Navigation Controller (`NavController`)

# Add dependencies

```
In build.gradle, under dependencies:

implementation "androidx.navigation:navigation-fragment-ktx:$nav_version"

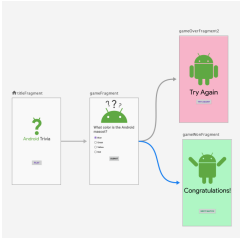
implementation "androidx.navigation:navigation-ui-ktx:$nav_version"
```

# Navigation host (NavHost)

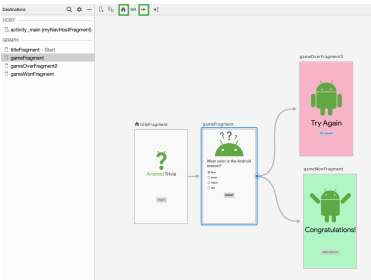
```
<fragment
 android:id="@+id/nav_host"
 android:name="androidx.navigation.fragment.NavHostFragment"
 android:layout_width="match_parent"
 android:layout_height="match_parent"
 app:defaultNavHost="true"
 app:navGraph="@navigation/nav_graph_name"/>
```

# Navigation graph

- New resource type located in res/navigation directory
- XML file containing all of your navigation destinations and actions
  - Lists all the (Fragment/Activity) destinations that can be navigated to
  - Lists the associated actions to traverse between them
  - Optionally lists animations for entering or exiting



# Navigation Editor in Android Studio



# Creating a Fragment

- Extend Fragment class
- Override onCreateView()
- Inflate a layout for the Fragment that you have defined in XML

```
class DetailFragment : Fragment() {

 override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?,
 savedInstanceState: Bundle?): View? {
 return inflater.inflate(R.layout.detail_fragment, container, false)
 }
}
```

# Specifying Fragment destinations

- Fragment destinations are denoted by the action tag in the navigation graph.
- Actions can be defined in XML directly or in the Navigation Editor by dragging from source to destination.
- Autogenerated action IDs take the form of action\_<sourceFragment>\_to\_<destinationFragment>.

# Example fragment destination

```
<fragment
 android:id="@+id/welcomeFragment"
 android:name="com.example.android.navigation.WelcomeFragment"
 android:label="fragment_welcome"
 tools:layout="@layout/fragment_welcome" >

 <action
 android:id="@+id/action_welcomeFragment_to_detailFragment"
 app:destination="@id/detailFragment" />

</fragment>
```

# Navigation Controller (NavController)

- NavController manages UI navigation in a navigation host.
- Specifying a destination path only names the action, but it doesn't execute it.
  - To follow a path, use NavController.

# Example NavController

```
class MainActivity : AppCompatActivity() {

 override fun onCreate(savedInstanceState: Bundle?) {
 ...
 val navController = findNavController(R.id.myNavHostFragment)
 }

 fun navigateToDetail() {
 navController.navigate(R.id.action_welcomeFragment_to_detailFragment)
 }
}
```

# More custom navigation behavior

## Passing data between destinations

Using Safe Args:

- Ensures arguments have a valid type
- Lets you provide default values
- Generates a `<SourceDestination>Directions` class with methods for every action in that destination
- Generates a class to set arguments for every named action
- Generates a `<TargetDestination>Args` class providing access to the destination's arguments

## Setting up Safe Args

In the project `build.gradle` file:

```
buildscript {
 repositories {
 google()
 }
 dependencies {
 classpath "androidx.navigation:navigation-safe-args-gradle-plugin:$nav_version"
 }
}
```

In the app's or module's `build.gradle` file:

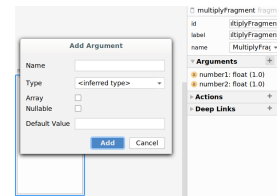
```
apply plugin: "androidx.navigation.safeargs.kotlin"
```

## Sending data to a Fragment

1. Create arguments the destination fragment will expect.
2. Create action to link from source to destination.
3. Set the arguments in the action method on `<Source>FragmentDirections`.
4. Navigate according to that action using the Navigation Controller.
5. Retrieve the arguments in the destination fragment.

## Destination arguments

```
<fragment
 android:id="@+id/multiplyFragment"
 android:name="com.example.arithmetic.MultiplyFragment"
 android:label="MultiplyFragment" >
 <argument
 android:name="number1"
 app:argType="float"
 android:defaultValue="1.0" />
 <argument
 android:name="number2"
 app:argType="float"
 android:defaultValue="1.0" />
</fragment>
```



## Supported argument types

Type	Type Syntax app:argType=<type>	Supports Default Values	Supports Null Values
Integer	"integer"	Yes	No
Float	"float"	Yes	No
Long	"long"	Yes	No
Boolean	"boolean"	Yes ("true" or "false")	No
String	"string"	Yes	Yes
Array	above type + "[]" (for example, "string[]" "long[]")	Yes (only "@null")	Yes
Enum	Fully qualified name of the enum	Yes	No
Resource reference	"reference"	Yes	No

## Supported argument types: Custom classes

Type	Type Syntax app:argType=<type>	Supports Default Values	Supports Null Values
Serializable	Fully qualified class name	Yes (only "@null")	Yes
Parcelable	Fully qualified class name	Yes (only "@null")	Yes

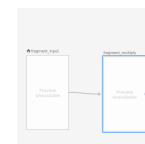
## Create action from source to destination

In `nav_graph.xml`:

```
<fragment
 android:id="@+id/fragment_input"
 android:name="com.example.arithmetic.InputFragment" >

 <action
 android:id="@+id/action_to_multiplyFragment"
 app:destination="@id/multiplyFragment" />

</fragment>
```



## Navigating with actions

In `InputFragment.kt`:

```
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
 super.onViewCreated(view, savedInstanceState)
 binding.button.setOnClickListener {
 val n1 = binding.number1.text.toString().toFloatOrNull() ?: 0.0
 val n2 = binding.number2.text.toString().toFloatOrNull() ?: 0.0

 val action = InputFragmentDirections.actionToMultiplyFragment(n1, n2)
 view.findNavController().navigate(action)
 }
}
```

# Retrieving Fragment arguments

```
class MultiplyFragment : Fragment() {
 val args: MultiplyFragmentArgs by navArgs()
 lateinit var binding: FragmentMultiplyBinding
 override fun onCreateView(view: View, savedInstanceState: Bundle?) {
 super.onCreateView(view, savedInstanceState)
 val number1 = args.number1
 val number2 = args.number2
 val result = number1 * number2
 binding.output.text = "${number1} * ${number2} = ${result}"
 }
}
```

# Navigation UI

# Menus revisited

```
override fun onOptionsItemSelected(item: MenuItem): Boolean {
 val navController = findNavController(R.id.nav_host_fragment)
 return item.onNavDestinationSelected(navController) ||
 super.onOptionsItemSelected(item)
}
```

# DrawerLayout for navigation drawer

```
<androidx.drawerlayout.widget.DrawerLayout
 android:id="@+id/drawer_layout" ...>

 <fragment
 android:name="androidx.navigation.fragment.NavHostFragment"
 android:id="@+id/nav_host_fragment" ... />

 <com.google.android.material.navigation.NavigationView
 android:id="@+id/nav_view"
 app:menu="@menu/activity_main_drawer" ... />

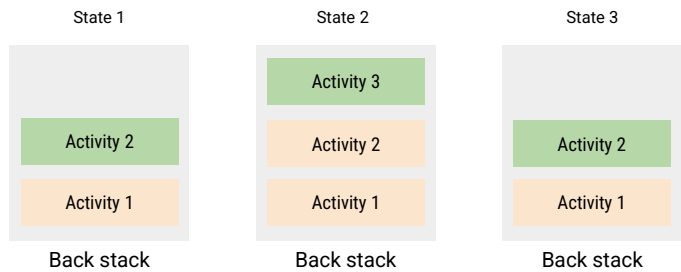
</androidx.drawerlayout.widget.DrawerLayout>
```

# Finish setting up navigation drawer

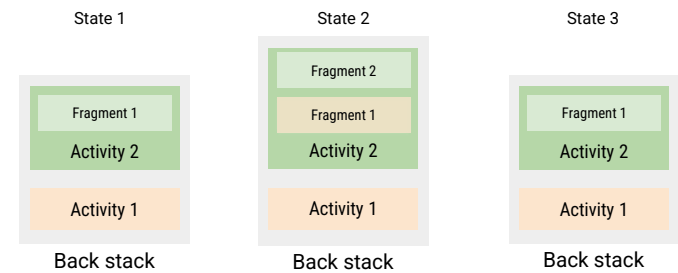
```
Connect DrawerLayout to navigation graph:
val appBarConfiguration = AppBarConfig(navController.graph, drawer)

Set up NavigationView for use with a NavController:
val navView = findViewById<NavigationView>(R.id.nav_view)
navView.setupWithNavController(navController)
```

# Understanding the back stack



# Fragments and the back stack



# Summary

# Summary

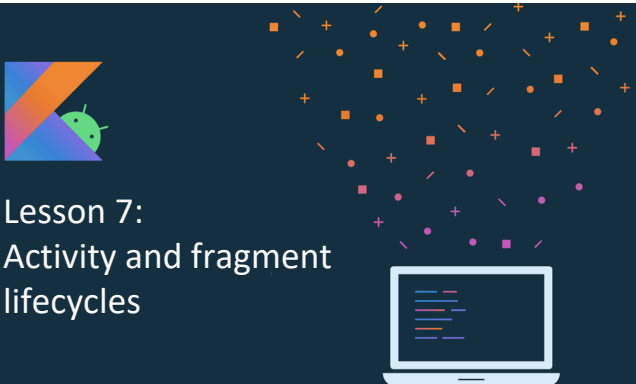
- In Lesson 6, you learned how to:
- Use explicit and implicit intents to navigate between Activities
  - Structure apps using fragments instead of putting all UI code in the Activity
  - Handle navigation with NavGraph, NavHost, and NavController
  - Use Safe Args to pass data between fragment destinations
  - Use NavigationUI to hook up top app bar, navigation drawer, and bottom navigation
  - Android keeps a back stack of all the destinations you've visited, with each new destination being pushed onto the stack.

# Learn more

- [Principles of navigation](#)
- [Navigation component](#)
- [Pass data between destinations](#)
- [NavigationUI](#)

# Pathway

Practice what you've learned by completing the pathway:  
[Lesson 6: App navigation](#)



## Lesson 7: Activity and fragment lifecycles

# About this lesson

Lesson 7: Activity and fragment lifecycles

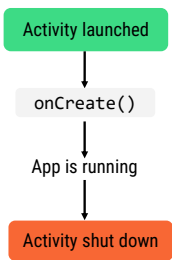
- [Activity lifecycle](#)
- [Logging](#)
- [Fragment lifecycle](#)
- [Lifecycle-aware components](#)
- [Tasks and back stack](#)
- [Summary](#)

# Activity lifecycle

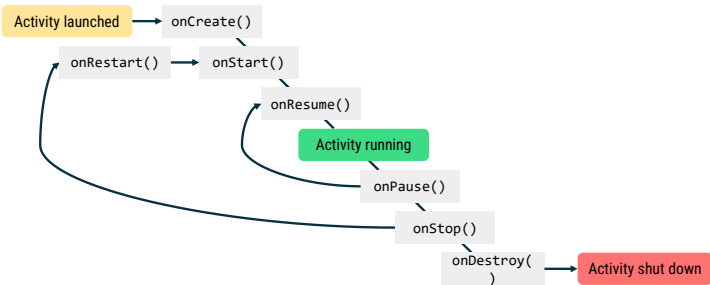
# Why it matters

- Preserve user data and state if:
  - User temporarily leaves app and then returns
  - User is interrupted (for example, a phone call)
  - User rotates device
- Avoid memory leaks and app crashes.

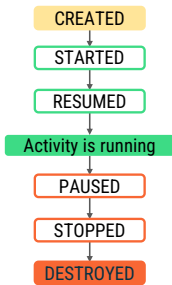
# Simplified activity lifecycle



# Activity lifecycle



# Activity states



## onCreate()

- Activity is created and other initialization work occurs
- You must implement this callback
- Inflate activity UI and perform other app startup logic

## onStart()

- Activity becomes visible to the user
- Called after activity:
  - `onCreate()`
  - or
  - `onRestart()` if activity was previously stopped

## onResume()

- Activity gains input focus:
  - User can interact with the activity
- Activity stays in resumed state until system triggers activity to be paused

## onPause()

- Activity has lost focus (not in foreground)
- Activity is still visible, but user is not actively interacting with it
- Counterpart to `onResume()`

## onStop()

- Activity is no longer visible to the user
- Release resources that aren't needed anymore
- Save any persistent state that the user is in the process of editing so they don't lose their work

## onDestroy()

- Activity is about to be destroyed, which can be caused by:
  - Activity has finished or been dismissed by the user
  - Configuration change
- Perform any final cleanup of resources.
- Don't rely on this method to save user data (do that earlier)

## Summary of activity states

State	Callbacks	Description
Created	<code>onCreate()</code>	Activity is being initialized.
Started	<code>onStart()</code>	Activity is visible to the user.
Resumed	<code>onResume()</code>	Activity has input focus.
Paused	<code>onPause()</code>	Activity does not have input focus.
Stopped	<code>onStop()</code>	Activity is no longer visible.
Destroyed	<code>onDestroy()</code>	Activity is destroyed.

## Save state

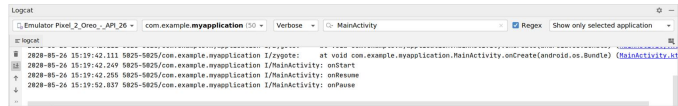
User expects UI state to stay the same after a config change or if the app is terminated when in the background.

- Activity is destroyed and restarted, or app is terminated and activity is started.
- Store user data needed to reconstruct app and activity Lifecycle changes:
  - Use `Bundle` provided by `onSaveInstanceState()`.
  - `onCreate()` receives the `Bundle` as an argument when activity is created again.

## Logging

# Logging in Android

- Monitor the flow of events or state of your app.
- Use the built-in `Log` class or third-party library.
- Example `Log` method call: `Log.d(TAG, "Message")`

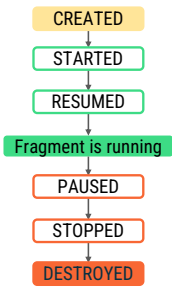


# Write logs

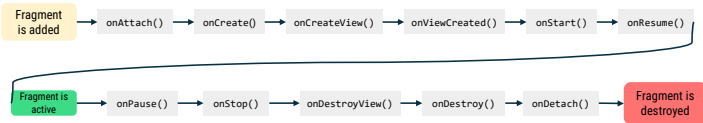
Priority level	Log method
Verbose	<code>Log.v(String, String)</code>
Debug	<code>Log.d(String, String)</code>
Info	<code>Log.i(String, String)</code>
Warning	<code>Log.w(String, String)</code>
Error	<code>Log.e(String, String)</code>

# Fragment lifecycle

## Fragment states



## Fragment lifecycle diagram



## onAttach()

- Called when a fragment is attached to a context
- Immediately precedes `onCreate()`

## onCreateView()

- Called to create the view hierarchy associated with the fragment
- Inflate the fragment layout here and return the root view

## onViewCreated()

- Called when view hierarchy has already been created
- Perform any remaining initialization here (for example, restore state from `Bundle`)

## onDestroyView() and onDetach()

- `onDestroyView()` is called when view hierarchy of fragment is removed.
- `onDetach()` is called when fragment is no longer attached to the host.

# Summary of fragment states

State	Callbacks	Description
Initialized	<code>onAttach()</code>	Fragment is attached to host.
Created	<code>onCreate()</code> , <code>onCreateView()</code> , <code>onViewCreated()</code>	Fragment is created and layout is being initialized.
Started	<code>onStart()</code>	Fragment is started and visible.
Resumed	<code>onResume()</code>	Fragment has input focus.
Paused	<code>onPause()</code>	Fragment no longer has input focus.
Stopped	<code>onStop()</code>	Fragment is not visible.
Destroyed	<code>onDestroyView()</code> , <code>onDestroy()</code> , <code>onDetach()</code>	Fragment is removed from host.

# Save fragment state across config changes

Preserve UI state in fragments by storing state in `Bundle`:

- `onSaveInstanceState(outState: Bundle)`

Retrieve that data by receiving the `Bundle` in these fragment callbacks:

- `onCreate()`
- `onCreateView()`
- `onViewCreated()`

# Lifecycle-aware components

## Lifecycle-aware components

Adjust their behavior based on activity or fragment lifecycle

- Use the `androidx.lifecycle` library
- `Lifecycle` tracks the lifecycle state of an activity or fragment
  - Holds current lifecycle state
  - Dispatches lifecycle events (when there are state changes)

## LifecycleOwner

- Interface that says this class has a lifecycle
  - Implementers must implement `getLifecycle()` method
- Examples: `Fragment` and `AppCompatActivity` are implementations of `LifecycleOwner`

## LifecycleObserver

Implement `LifecycleObserver` interface:

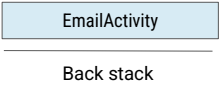
```
class MyObserver : LifecycleObserver {
 @OnLifecycleEvent(Lifecycle.Event.ON_RESUME)
 fun connectListener() {
 ...
 }
}
```

Add the observer to the lifecycle:

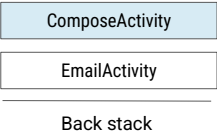
```
myLifecycleOwner.getLifecycle().addObserver(MyObserver())
```

# Tasks and back stack

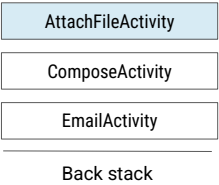
## Back stack of activities



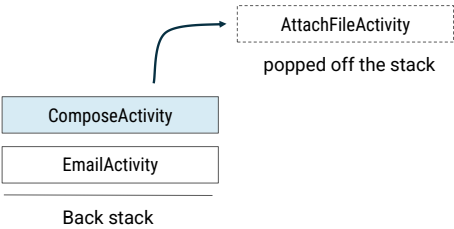
## Add to the back stack



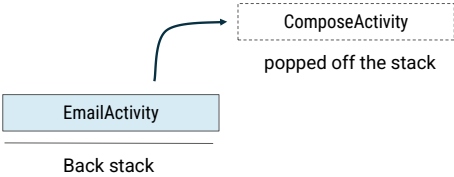
## Add to the back stack again



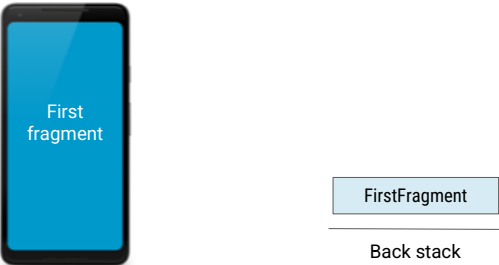
## Tap Back button



## Tap Back button again



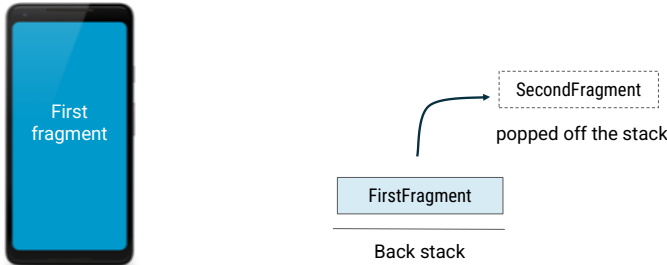
## First destination in the back stack



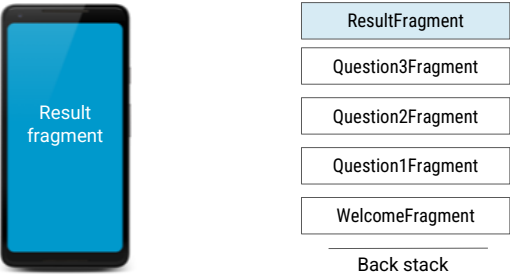
## Add a destination to the back stack



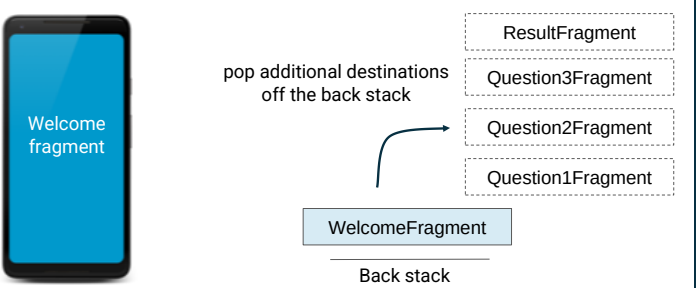
## Tap Back button



## Another back stack example



## Modify Back button behavior



# Summary

## Summary

In Lesson 7, you learned how to:

- Understand how an activity instance transitions through different lifecycle states as the user interacts with or leaves your app
- Reserve UI state across configuration changes using a `Bundle`
- Fragment lifecycle callback methods similar to activity, but with additions
- Use lifecycle-aware components help organize your app code
- Use default or custom back stack behavior
- Use logging to help debug and track the state of the app

## Learn more

- [Understand the Activity Lifecycle](#)
- [Activity class](#)
- [Fragments guide and lifecycle](#)
- [Fragment class](#)

## Pathway

Practice what you've learned by completing the pathway:

[Lesson 7: Activity and Fragment Lifecycles](#)

